

Assignment 6 – LCD-display

ING1507 – Mikrokontrollere

Oskar Hellebust-Nygaard

March 26, 2026

Contents

1	Introduksjon	2
2	Teori	2
2.1	LCD	2
2.2	8-bit modus	2
2.3	Viktige kommandoer	2
3	Kobling	3
4	Implementasjon	4
4.1	lcd.h	4
4.2	main.c	6
4.3	Forklaring av funksjonene	7
5	Feilsøking og problemer underveis	7
5.1	Skjerm lyser, men ingen tekst	8
5.2	Bytting av potmeter	8
5.3	Bytting av LCD	8
5.4	Baklys	8
5.5	Det faktiske problemet	9
6	Resultat	9

1 Introduksjon

Oppgaven går ut på å bruke et bibliotek for en 16×2 LCD-modul, og bruke dette til å vise fødested og fødselsdato på skjermen. Biblioteket skal legges i en header-fil (`lcd.h`) og inneholde `send command`, `send character`, `initialize LCD`, `send string`, `send number`, `send string with location`, `send number with location` og `clear screen`.

2 Teori

2.1 LCD

En 16×2 LCD-modul styres av HD44780-kontrolleren. Kontrolleren har to typer minne som er relevante her: DDRAM (Display Data RAM), som lagrer tegnene som vises på skjermen, og CGROM, som inneholder de innebygde tegnene.

Kommunikasjonen mellom mikrokontrolleren og LCD-en skjer via tre kontrollpinner og en 8-bits (eller 4-bits) databus:

- **RS (Register Select):** $RS = 0$ betyr at databusen inneholder en kommando. $RS = 1$ betyr at den inneholder et tegn som skal skrives til DDRAM.
- **RW (Read/Write):** $RW = 0$ betyr skriv til LCD. $RW = 1$ betyr les fra LCD. I denne implementasjonen brukes kun skriving.
- **E (Enable):** LCD-en leser ikke busen kontinuerlig. Den leser kun når E-pinnen mottar en kort puls (høy $\rightarrow$ lav). Dette heter enable-wait-disable-wait.

2.2 8-bit modus

I 8-bit modus sendes hele kommando- eller tegnbyte på en gang via DB0–DB7. Dette krever 11 ledninger totalt (RS, RW, E + 8 datapinner), men er enklere å implementere enn 4-bit modus og raskere ved samme klokke.

2.3 Viktige kommandoer

Tabell 1 viser kommandoene brukt i initialiseringen.

Table 1: LCD-kommandoer brukt i oppgaven

Kommando	Hex	Beskrivelse
Function set	0x38	8-bit modus, 2 linjer, 5×8 font
Display ON	0x0C	Display på, markør av, blinking av
Clear display	0x01	Sletter DDRAM og returnerer markør til hjemposisjon
Entry mode	0x06	Markør beveger seg til høyre etter hvert tegn
Set DDRAM	0x80 + <code>addr</code>	Setter markørposisjon (linje 1: 0x00–0x0F, linje 2: 0x40–0x4F)

3 Kobling

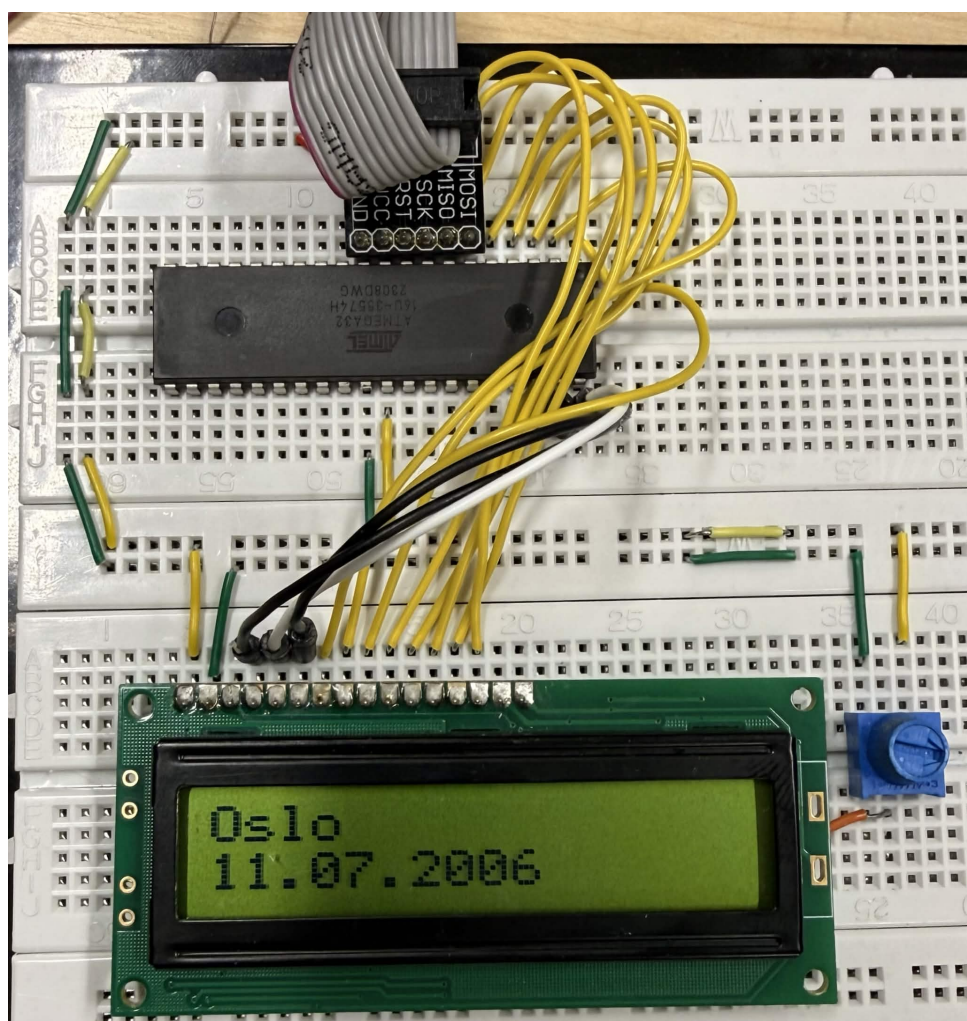


Figure 1: Koblingsskjema ATmega32 – LCD

Kobling mellom ATmega32 og LCD-modulen:

Table 2: Pinnekobling ATmega32 – LCD

LCD-pinne	Funksjon	ATmega32
Pin 1 (VSS)	GND	GND
Pin 2 (VDD)	+5V	+5V
Pin 3 (V0)	Kontrast	Potmeter midtpinne
Pin 4 (RS)	Register select	PA0
Pin 5 (RW)	Read/Write	PA1
Pin 6 (E)	Enable	PA2
Pin 7 (DB0)	Data bit 0	PB0
Pin 8 (DB1)	Data bit 1	PB1
Pin 9 (DB2)	Data bit 2	PB2
Pin 10 (DB3)	Data bit 3	PB3
Pin 11 (DB4)	Data bit 4	PB4
Pin 12 (DB5)	Data bit 5	PB5
Pin 13 (DB6)	Data bit 6	PB6
Pin 14 (DB7)	Data bit 7	PB7
Pin 15 (LED+)	Baklys +	Ikke koblet
Pin 16 (LED-)	Baklys -	Ikke koblet
AVCC	ADC-forsyning	+5V

Port A brukes til kontrollpinner (RS, RW, E) og Port B brukes som 8-bits databuss. AVCC på ATmega32 (fysisk pin 30) må kobles til +5V siden Port A er knyttet til ADC-forsyningen, selv når ADC ikke er i bruk.

4 Implementasjon

4.1 lcd.h

Listing 1: lcd.h – komplett header-fil

```
1 #ifndef LCD_H_
2 #define LCD_H_
3
4 #define F_CPU 1000000UL
5 #include <avr/io.h>
6 #include <util/delay.h>
7 #include <stdlib.h>
8
9 /* Kontrollpinner pa PORTA */
10 #define RS PA0
11 #define RW PA1
12 #define E PA2
13 #define DDRCommand DDRA
14 #define PORTCommand PORTA
15
```

```

16  /* Datapinner pa PORTB */
17  #define DDRData      DDRB
18  #define PORTData     PORTB
19
20  /* Enable-wait-disable-wait */
21  static void LCD_enable(void)
22  {
23      PORTCommand |= (1 << E);
24      _delay_us(1);
25      PORTCommand &= ~(1 << E);
26      _delay_us(1530);
27  }
28
29  /* Send kommando */
30  static void LCD_command(unsigned char cmd)
31  {
32      PORTCommand &= ~(1 << RS); /* RS = 0 -> kommando */
33      PORTCommand &= ~(1 << RW); /* RW = 0 -> skriv */
34      _delay_us(1);
35      PORTData = cmd;
36      _delay_us(1);
37      LCD_enable();
38  }
39
40  /* Send tegn */
41  static void LCD_char(unsigned char data)
42  {
43      PORTCommand |= (1 << RS); /* RS = 1 -> data */
44      PORTCommand &= ~(1 << RW); /* RW = 0 -> skriv */
45      _delay_us(1);
46      PORTData = data;
47      _delay_us(1);
48      LCD_enable();
49  }
50
51  /* Initialisering */
52  static void LCD_init(void)
53  {
54      DDRCCommand |= (1 << RS) | (1 << RW) | (1 << E);
55      DDRData      = 0xFF;
56
57      _delay_ms(50); /* power-on delay */
58
59      LCD_command(0x38); /* 8-bit, 2 linjer, 5x8 font */
60      LCD_command(0x0C); /* display pa, markur av */
61      LCD_command(0x01); /* clear display */
62      _delay_ms(2); /* clear tar 1.53 ms */
63      LCD_command(0x06); /* entry mode: cursor til hoyre */
64  }
65
66  /* Skriv streng */

```

```

67 static void LCD_string(const char *str)
68 {
69     while (*str)
70         LCD_char(*str++);
71 }
72
73 /* Skriv tall */
74 static void LCD_number(int number)
75 {
76     char buffer[16];
77     itoa(number, buffer, 10);
78     LCD_string(buffer);
79 }
80
81 /* Flytt markur */
82 static void LCD_goto(unsigned char row, unsigned char col)
83 {
84     if (row == 0)
85         LCD_command(0x80 + col);
86     else
87         LCD_command(0xC0 + col);
88 }
89
90 /* Skriv streng pa valgt plass */
91 static void LCD_string_xy(unsigned char row, unsigned char col,
92                          const char *str)
93 {
94     LCD_goto(row, col);
95     LCD_string(str);
96 }
97
98 /* Skriv tall pa valgt plass */
99 static void LCD_number_xy(unsigned char row, unsigned char col,
100                          int number)
101 {
102     LCD_goto(row, col);
103     LCD_number(number);
104 }
105
106 /* Tom skjermen */
107 static void LCD_clear(void)
108 {
109     LCD_command(0x01);
110     _delay_ms(2);
111 }
112
113 #endif

```

4.2 main.c

Listing 2: main.c – viser fødested og fødselsdato

```
1 #define F_CPU 1000000UL
2 #include <avr/io.h>
3 #include "lcd.h"
4
5 int main(void)
6 {
7     LCD_init();
8
9     LCD_string_xy(0, 0, "Oslo");
10    LCD_string_xy(1, 0, "11.07.2006");
11
12    while (1)
13    {
14    }
15 }
```

4.3 Forklaring av funksjonene

LCD_enable() implementerer enable-wait-disable-wait-mønsteret fra leksjonen. LCD-kontrolleren leser ikke databussen kontinuerlig, men kun når E-pinnen mottar en falling edge. Etter pulsen ventes det 1530 μ s slik at kontrolleren rekker å prosessere kommandoen.

LCD_command() setter RS=0 og RW=0 for å indikere en kommando i skriveretning. Data legges på PORTB, og deretter trigges enable-pulsen.

LCD_char() fungerer likt, men med RS=1 siden dette er tegn som skal skrives til DDRAM.

LCD_init() setter portretningene, venter 50 ms etter power-on, og sender deretter de fire nødvendige oppstartskommandoene. Det legges til en eksplisitt 2 ms delay etter clear display siden denne kommandoen ifølge databladet tar 1.53 ms – markert lengre enn de andre.

LCD_goto() beregner DDRAM-adressen. Linje 0 starter på adresse 0x00 og linje 1 starter på 0x40. Kommandoen 0x80 kombinert med adressen setter markøren.

LCD_string() itererer tegn for tegn til null-terminatoren `\0`.

LCD_number() konverterer et heltall til tekst med `itoa()` fra `stdlib.h`, siden LCD-en kun kan motta ASCII-tegn.

5 Feilsøking og problemer underveis

Arbeidet med denne oppgaven krevde mye feilsøking. Her er de problemene som oppstod, hva vi trodde var årsaken, og hva som faktisk løste det.

5.1 Skjerm lyser, men ingen tekst

Symptom: Bakgrunnslyset var synlig og kontrasten lot seg justere med potmeteret, men ingen tegn viste seg på skjermen.

Mulige årsaker som ble undersøkt:

1. **Feil pinnekobling** – I starten var kontrollpinnene (RS, RW, E) koblet til Port C (PC0, PC1, PC2). På ATmega32 er PC2–PC5 reservert av JTAG-grensesnittet, som forstyrrer normal digital I/O på disse pinnene. E-pinnen på PC2 kunne dermed ikke sende gyldige enable-pulser. Løsningsforsøk: JTAG deaktiveres ved å skrive JTD-bit to ganger til MCUCSR. Dette hjalp ikke, og kontrollpinnene ble flyttet til Port A (PA0–PA2) i stedet, noe som eliminerte JTAG-problemet.
2. **For kort power-on delay** – Init-koden hadde `_delay_us(1530)` som oppstarts-delay. HD44780-datasheeten krever minimum 15 ms etter spenning på. 1530 μ s er kun 1,53 ms – omtrent ti ganger for lite. Delay ble økt til 50 ms.
3. **For kort delay etter clear display** – Clear display-kommandoen (0x01) tar 1,53 ms ifølge databladet, markert lengre enn de fleste andre kommandoer (39 μ s). Uten eksplisitt delay etter denne ble neste kommando sendt mens kontrolleren ennå ikke var ferdig med å slette skjermen.
4. **AVCC ikke koblet** – Port A på ATmega32 er koblet til ADC-forsyningsspenningen AVCC. Hvis AVCC ikke kobles til +5V, fungerer ikke Port A som forventet selv om ADC ikke er i bruk. AVCC ble verifisert koblet.
5. **RS/RW satt etter data på bussen** – I en tidlig versjon av koden ble PORTData satt før RS og RW, noe som potensielt kunne føre til at LCD-en leste data med feil kontrollsignaler. Rekkefølgen ble endret til: sett RS/RW $\rightarrow$ vent $\rightarrow$ sett data $\rightarrow$ vent $\rightarrow$ enable.

5.2 Bytting av potmeter

Potmeteret ble byttet ut for å utelukke at kontrastjusteringen var defekt. Dette endret ikke oppførselen til skjermen og var ikke årsaken til problemet.

5.3 Bytting av LCD

LCD-modulen ble byttet ut med en annen. Den første modulen hadde synlig pikseldegenerasjon, men begge modulene oppførte seg likt – ingen tekst. Dette bekreftet at problemet var i koden eller koblingen, ikke i maskinvaren.

5.4 Baklys

Baklyskoblingen ble testet ved å fjerne den. Dette hadde ingen effekt på om tekst viste seg.

5.5 Det faktiske problemet

Etter at alle fysiske og timing-relaterte problemer var rettet opp, viste skjermen fortsatt ingen tekst i en testøkt. Etter systematisk gjennomgang ble det oppdaget at LCD har eget ikke-flyktig DDRAM som *ikke nullstilles automatisk* når mikrokontrolleren programmeres på nytt eller resettes. Hvis clear display-kommandoen (0x01) ikke når frem korrekt under init – for eksempel fordi timing er feil – vil gammelt innhold fra en tidligere programkjøring ligge igjen i DDRAM. Skjermen viser da hverken nytt innhold eller blanke felter, men gammel minnestøy.

Løsning: Fysisk frakobling av LCD-strømmen slik at DDRAM tømmes helt, etterfulgt av korrekt init-sekvens med tilstrekkelige delays. Etter dette fungerte koden som forventet og teksten ble vist korrekt.

6 Resultat

Etter feilsøkingen fungerer programmet som tiltenkt. Skjermen viser:

```
Oslo    (linje 1, posisjon 0)
11.07.2006  (linje 2, posisjon 0)
```